Crypto IA-2

1)

a. BadStore's cart cookie is also an encoded string with a predictable structure XXX:YYY:... etc., and it probably contains information it shouldn't. Which field of the decoded string could an attacker change to give himself a discount on an item's price?
b. Using Scapy, create a frame that consists of an Ethernet layer, with an IP layer on top? With the show method, capture all fields of the frame.
c. Write a Shell script for port scan using NetCat.

(a) BadStore Cookie Manipulation

1) What is the experiment about

This experiment is about **BadStore web application security**.

It focuses on how attackers can **modify encoded cookie values** to gain unauthorized benefits like discounts.

From your PDF:

- BadStore contains vulnerabilities where user input and data (like cookies) are not properly secured
- Cookies may store sensitive values in encoded format (XXX:YYY...)

The idea:

An attacker decodes the cookie → modifies values → re-encodes → sends it back.

---

2) How to perform (steps)

1. Open **BadStore website in browser**
2. Login or add item to cart
3. Open **Developer Tools (Inspect → Application → Cookies)**
4. Locate **cart cookie**
5. Copy the cookie value
6. Decode the cookie string
7. Modify the required field
8. Re-encode and replace cookie
9. Refresh page

---

3)

i) Important concept (from PDF)

- Data is stored in format like: `XXX:YYY:...`
- Some values represent:
  - item
  - quantity
  - **price**

The vulnerable field is:

PRICE field

✔️ Attacker changes the **price value** to a lower number

✔️ This gives discount or even free items

---

ii) No program needed

(Only browser + inspect tool)

---

4) What to show in output

- Cookie value before modification
- Decoded string
- Modified value (price reduced)
- Final page showing reduced price

(b) Scapy Frame Creation

1) What is the experiment about

This experiment uses **Scapy** to create network packets.

From PDF:

- Scapy can create packets using layers like
  - Ethernet (Layer 2)
  - IP (Layer 3)

We create a **frame = Ethernet + IP**

---

2) How to perform (steps)

1. Open terminal
2. Type:

   ```
   scapy
   ```

   OR run Python file

---

3)

i) Program (from PDF)

```
fromscapy.allimport*

packet=IP()/TCP()
e_pack=Ether()/packet

e_pack.show()
```

---

ii) Explanation

- `IP()` → creates IP layer
- `TCP()` → transport layer
- `Ether()` → Ethernet layer
- `Ether()/packet` → combines layers
- `show()` → displays all fields

✔️ This prints complete frame details

---

4) What to show in output

- Output of:

  ```
  e_pack.show()
  ```

- It should display:
  - Source MAC
  - Destination MAC
  - IP fields
  - TCP fields

(c) Shell Script using Netcat

1) What is the experiment about

This experiment performs **port scanning using NetCat**.

From PDF:

- Netcat can scan ports using `nc -zv`
- Script automates scanning multiple ports

---

2) How to perform (steps)

1. Open terminal
2. Create file:

   ```
   nano port_scan.sh
   ```

3. Paste code
4. Save file
5. Give permission:

   ```
   chmod +x port_scan.sh
   ```

6. Run:

   ```
   ./port_scan.sh
   ```

---

3)

i) Script (from PDF)

```
#!/bin/bash

host="www.example.com"

read-p"Enter the starting port number: " start_port
read-p"Enter the ending port number: " end_port

for ((port=start_port; port<=end_port; port++ ))
do
echo"Scanning port$port..."
```

```
nc-zv-w1"$host""$port"2>&1
done
```

---

ii) Explanation

- `host` → target website
- `read` → takes input from user
- `for loop` → scans ports one by one
- `nc -zv` → checks if port is open
- `w1` → timeout 1 second
- `2>&1` → shows output properly

---

4) What to show in output

- Input:
  - Start port
  - End port
- Output:
  - List of ports
  - Open ports shown clearly

Example:

```
Scanning port 80...
Connection succeeded!
```

2)

a. Log in as joe@supplier.com to Badstore— — this is possible in a variety of ways, including SQL injection. Then look at his previous orders and answer the question: What credit card number did he use to make a purchase of $46.95? Multiple answers are possible.
b. How to Sniff with Scapy (for 10 packets and show the details of all the packets).

c. Using NetCat, show how to Communicate with the server using 2 terminals and across systems

(a) BadStore – Supplier Login & Credit Card

1) What is the experiment about

This experiment is about **BadStore vulnerabilities** like:

- SQL Injection
- Unauthorized access
- Viewing sensitive data (orders, credit card details)

From your PDF:

- You can login as supplier using extracted credentials
- Example: **joe@supplier.com**

---

2) How to perform (steps)

1. Open BadStore in browser
2. Use SQL Injection / admin login:
   - From PDF:
     - `admin : secret`
3. Go to:

   ```
   http://<ip>/cgi-bin/badstore.cgi?action=admin
   ```

4. Click **Show Current Users**
5. Find supplier account:
   - joe@supplier.com
6. Get password (hash → decode using MD5 site)
7. Login as:

```
joe@supplier.com
```

8. Go to **Supplier Portal**
9. Check **Previous Orders**

---

3)

i) Important concept

- BadStore stores sensitive info like:
  - user data
  - passwords
  - **credit card details**

👉 By accessing supplier account:

- You can view order history
- Extract credit card info

---

ii) Answer (from experiment)

✔️ The **credit card number** used for purchase of **$46.95**:

👉 **Multiple answers possible (as mentioned in PDF)**

👉 Write any valid number shown in your output screen

---

4) What to show in output

- Login as supplier
- Supplier dashboard
- Previous orders page
- Order with **$46.95**
- Credit card number visible

✅ (b) Scapy – Sniff 10 Packets

1) What is the experiment about

This experiment uses **Scapy to capture (sniff) network packets**.

From PDF:

- `sniff(count=10)` captures 10 packets

---

2) How to perform (steps)

1. Open terminal
2. Run Python / Scapy
3. Execute code

---

3)

i) Program (from PDF)

```
fromscapy.allimport*

packets=sniff(count=10)

packets.summary()

print(packets[0].show())
print(packets[1].show())
```

---

ii) Explanation

- `sniff(count=10)` → captures 10 packets
- `summary()` → brief info
- `packets[i].show()` → full details
- Index `[0], [1]` → access packets

✔️ Shows:

- Source IP
- Destination IP

- Protocol
- Packet fields

---

4) What to show in output

- Summary of 10 packets
- Detailed fields using `.show()`
- Wireshark verification (as PDF says)

✅ (c) Netcat – Communication (2 terminals)

1) What is the experiment about

This experiment shows **Netcat communication**:

- Between terminals
- Between systems

From PDF:

- Netcat works as client-server communication tool

---

2) How to perform (steps)

👉 Same Machine

1. Open Terminal 1:

```
nc -vlp 1234
```

1. Open Terminal 2:

```
nc localhost 1234
```

1. Type messages → press Enter

👉 Across Systems

1. On Machine A:

```
nc -vlp 1234
```

1. On Machine B:

```
nc <IP_of_A> 1234
```

1. Start chatting

---

3)

i) Commands + explanation

- `nc -vlp 1234`
  - `v` → verbose
  - `l` → listen
  - `p` → port
- `nc localhost 1234`
  - connects to server

---

ii) No program needed

(only commands)

---

4) What to show in output

- Terminal 1 listening
- Terminal 2 connected
- Messages typed
- Messages appearing in both terminals

3)

a. Demonstrate how to perform traceroute in Scapy.
b. Using NetCat, show the usage of HTML Request/Response (HEAD /GET).
c. Using Nmap perform Ping Scan (determine which hosts are up). Also show how to scan a single IP.

(a) Scapy – Traceroute

1) What is the experiment about

This experiment demonstrates **Traceroute using Scapy**.

From your PDF:

- Scapy can trace the path of packets across network
- It shows intermediate routers (hops)

---

2) How to perform (steps)

1. Open terminal
2. Start Python / Scapy:

```
scapy
```

3. Run traceroute command

---

3)

i) Program (from PDF)

```
fromscapy.allimport*

result,unanswered=traceroute(["www.example.com","www.googl
e.com"],maxttl=20)

result.show()
```

---

ii) Explanation

- `traceroute()` → finds path to destination
- `maxttl=20` → maximum hops
- Returns:
  - `result` → successful responses
  - `unanswered` → no response
- `result.show()` → displays full details

✔️ Shows:

- Each hop IP
- Delay/time
- Path to destination

---

4) What to show in output

- List of hops
- IP addresses of routers
- Final destination reached

✅ (b) Netcat – HTML Request/Response (GET / HEAD)

1) What is the experiment about

This experiment shows how **Netcat communicates with web server using HTTP requests**.

From your PDF:

- You manually send:
  - GET request
  - HEAD request

---

2) How to perform (steps)

1. Open terminal
2. Connect to server:

```
nc example.com 80
```

---

3)

i) Commands + explanation

🔹 GET Request

Type inside terminal:

```
GET / HTTP/1.1
Host: example.com
```

👉 Press Enter twice

✔️ This sends request for webpage

🔹 HEAD Request

```
HEAD / HTTP/1.1
Host: example.com
```

👉 Press Enter twice

✔️ This requests only headers

---

ii) Explanation

- `nc example.com 80` → connects to HTTP server
- `GET` → fetch full webpage
- `HEAD` → fetch only headers
- `Host` → required in HTTP/1.1

---

4) What to show in output

- For GET:
  - HTTP response + HTML content
- For HEAD:
  - HTTP headers only

Example:

```
HTTP/1.1 200 OK
Content-Type: text/html
```

✅ (c) Nmap – Ping Scan & Single IP Scan

1) What is the experiment about

This experiment uses **Nmap for network scanning**.

From your PDF:

- Ping scan → find live hosts
- IP scan → scan ports of one system

---

2) How to perform (steps)

1. Open terminal
2. Run Nmap commands

---

3)

i) Commands + explanation

🔹 Ping Scan

```
nmap -sn 192.168.1.0/24
```

✔️ From PDF:

- Detects live hosts

- No port scanning

👉 `-sn` = Ping scan

🔹 Scan Single IP

```
nmap 192.168.1.1
```

✔️ Performs basic port scan

---

ii) Explanation

- Ping scan:
  - Uses ICMP / ARP
  - Finds active devices
- Single IP scan:
  - Shows open ports
  - Shows services

---

4) What to show in output

Ping Scan:

- List of live hosts

```
Host is up
```

Single IP:

- Open ports

```
PORT    STATE SERVICE
80      open  http
```

4)

a. Write a python code using Scapy for creating a Network Scanner (Hint: Create an ARP packet using ARP() method)

b. Using NetCat, show communication between browser and the server. (for plain text)
c. Using Nmap, Scan a range of IPs. Also scan an entire subnet.

(a) Scapy – Network Scanner (ARP)

1) What is the experiment about

This experiment creates a **network scanner using Scapy**.

From your PDF:

- Uses **ARP packets** to discover devices in a network
- Prints **IP and MAC addresses**

---

2) How to perform (steps)

1. Open terminal
2. Run Python / Scapy script
3. Execute the program

---

3)

i) Program (from PDF)

```
importscapy.allasscapy

request=scapy.ARP()
request.pdst="192.168.5.13/24"

broadcast=scapy.Ether()
broadcast.dst="ff:ff:ff:ff:ff:ff"

request_broadcast=broadcast/request

clients=scapy.srp(request_broadcast,timeout=1)[0]
```

```
forelementinclients:
print(element[1].psrc+" - "+element[1].hwsrc)
```

---

ii) Explanation

- `ARP()` → creates ARP request
- `pdst` → target IP range
- `Ether()` → Ethernet frame
- `ff:ff:ff:ff:ff:ff` → broadcast address
- `/` → combines packets
- `srp()` → send & receive packets
- `psrc` → IP address
- `hwsrc` → MAC address

✔️ Finds all devices in network

---

4) What to show in output

- List of devices like:

```
192.168.5.1 - xx:xx:xx:xx:xx
192.168.5.2 - xx:xx:xx:xx:xx
```

✅ (b) Netcat – Browser & Server Communication (Plain Text)

1) What is the experiment about

This experiment shows how **Netcat acts as a simple HTTP server** and communicates with browser.

From your PDF:

- Browser sends request

- Netcat replies with HTTP response

---

2) How to perform (steps)

1. Open terminal
2. Start Netcat server:

```
nc -l -p 5000
```

1. Open browser:

```
http://localhost:5000
```

1. Browser sends request → appears in terminal

---

3)

i) Commands + explanation

In terminal (after browser connects), type:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Server: nooblinux

Hello from Netcat!
```

👉 Press Enter twice

---

ii) Explanation

- `nc -l -p 5000` → listen on port 5000
- Browser sends HTTP request
- You manually send response
- Double Enter → completes response

---

4) What to show in output

- Terminal showing request from browser
- Browser showing:

```
Hello from Netcat!
```

✅ (c) Nmap – Scan Range of IPs & Subnet

1) What is the experiment about

This experiment uses **Nmap to scan multiple systems**.

From your PDF:

- Scan range of IPs
- Scan entire subnet

---

2) How to perform (steps)

1. Open terminal
2. Run Nmap commands

---

3)

i) Commands + explanation

🔹 Scan Range of IPs

```
nmap 192.168.1.100-120
```

✔️ Scans IPs from 100 to 120

🔹 Alternative Range Format

```
nmap 192.168.1.5 192.168.1.10 192.168.1.15
```

🔹 Scan Entire Subnet

```
nmap 192.168.1.0/24
```

✔️ Scans all 256 IPs

---

ii) Explanation

- Range scan → multiple IPs
- Subnet `/24` → full network
- Finds:
  - live hosts
  - open ports

---

4) What to show in output

- List of scanned IPs
- Host status
- Open ports

Example:

```
Nmap scan report for 192.168.1.101
Host is up
PORT   STATE SERVICE
80     open  http
```

5)

a. BadStore uses cookies to track the contents of the cart, once you've added something to it. What is the key of the cookie used for the cart?
b. Write a Shell script for port scan using NetCat.
c. Using Nmap, scan top 1000 ports (default). Also show how to detect service version.

(a) BadStore – Cookie Key

1) What is the experiment about

This experiment is about **BadStore cookie handling**.

From your PDF:

- BadStore uses cookies to store cart information
- Cookies track items added to cart

---

2) How to perform (steps)

1. Open BadStore in browser
2. Add any item to cart
3. Right click → **Inspect**
4. Go to:
    - Application → Cookies
5. Find the cookie storing cart data

---

3)

i) Concept

- Cookie stores:
    - item details
    - quantity
    - price

👉 The **key used for cart cookie** is:

✅ cart

---

ii) No program required

---

4) What to show in output

- Cookie section in browser
- Cookie name = **cart**
- Value (encoded string)

✅ (b) Netcat – Shell Script (Port Scan)

1) What is the experiment about

This experiment performs **port scanning using Netcat script**.

From your PDF:

- Uses `nc -zv` inside loop

---

2) How to perform (steps)

1. Open terminal
2. Create script:

```
nano port_scan.sh
```

3. Paste code
4. Save
5. Give permission:

```
chmod +x port_scan.sh
```

6. Run:

```
./port_scan.sh
```

---

3)

i) Script (from PDF)

```
#!/bin/bash

host="www.example.com"

read-p"Enter the starting port number: " start_port
read-p"Enter the ending port number: " end_port

for ((port=start_port; port<=end_port; port++ ))
do
echo"Scanning port$port..."
nc-zv-w1"$host""$port"2>&1
done
```

---

ii) Explanation

- `read` → user input
- `for loop` → iterate ports
- `nc -zv` → check open ports
- `w1` → timeout

---

4) What to show in output

- Input ports
- Output showing open ports

✅ (c) Nmap – Top 1000 Ports & Service Version

1) What is the experiment about

This experiment uses **Nmap for port scanning and service detection**.

From your PDF:

- Default scan → top 1000 ports

- `sV` → service version detection

---

2) How to perform (steps)

1. Open terminal
2. Run commands

---

3)

i) Commands + explanation

🔹 Scan Top 1000 Ports (Default)

```
nmap 192.168.1.1
```

✔️ Default behaviour → scans top 1000 ports

🔹 Detect Service Version

```
nmap -sV 192.168.1.1
```

✔️ Shows:

- service name
- version

---

ii) Explanation

- Default scan → fast scanning
- `sV` → identifies service version

---

4) What to show in output

Default Scan:

```
PORT   STATE SERVICE
```

```
80      open   http
```

Service Version:

```
PORT    STATE SERVICE VERSION
80      open   http    Apache 2.4
```

6)

a. BadStore uses cookies to track the contents of the cart, once you've added something to it. What is the key of the cookie used for the cart?
b. Using Nmap show OS detection. Also demonstrate how to perform Aggressive scan.
c. One of the BadStore pages has a hidden form field that establishes a new user's privilege level. What is the name of this field?

(a) BadStore – Cookie Key

1) What is the experiment about

This experiment deals with **how BadStore uses cookies to store cart data**.

From your PDF:

- Cookies track items added to cart
- Stored in encoded format

---

2) How to perform (steps)

1. Open BadStore website
2. Add item to cart
3. Right click → **Inspect**
4. Go to:
    - Application → Cookies

5. Observe cookie list

---

3)

i) Concept

👉 The cookie used for storing cart data has key:

✅ cart

- Stores:
  - item
  - quantity
  - price

---

ii) No program required

---

4) What to show in output

- Cookie section
- Cookie name = **cart**
- Encoded value

✅ (b) Nmap – OS Detection & Aggressive Scan

1) What is the experiment about

This experiment uses **Nmap advanced scanning techniques**.

From your PDF:

- `O` → OS detection
- `A` → Aggressive scan

---

2) How to perform (steps)

1. Open terminal
2. Run Nmap commands

---

3)

i) Commands + explanation

🔹 OS Detection

```
nmap -O 192.168.1.1
```

✔️ Detects:

- Operating system
- Device type

🔹 Aggressive Scan

```
nmap -A 192.168.1.1
```

✔️ Combines:

- OS detection
- Service version (`sV`)
- Script scanning
- Traceroute

---

ii) Explanation

- `O` → identifies OS
- `A` → full detailed scan

---

4) What to show in output

OS Detection:

```
OS details: Linux / Windows
```

Aggressive Scan:

```
PORT   STATE SERVICE VERSION
80     open  http    Apache
OS details: Linux
```

✅ (c) BadStore – Hidden Field

1) What is the experiment about

This experiment shows **hidden fields vulnerability in BadStore**.

From your PDF:

- Hidden fields can control user privilege
- Can be viewed using inspect tool

---

2) How to perform (steps)

1. Login as admin:

   ```
   admin : secret
   ```

2. Go to:

   ```
   /cgi-bin/badstore.cgi?action=admin
   ```

3. Click **Add User**
4. Right click → Inspect
5. Press **Ctrl + F** → search hidden field

---

3)

i) Answer (from PDF)

👉 The hidden field name is:

✅ password

✔️ Found in input field:

```
<inputtype="hidden"name="password">
```

---

ii) Explanation

- Hidden field is not visible
- But accessible via inspect
- Used to control user privilege

---

4) What to show in output

- Admin panel
- Add user page
- Inspect view showing hidden field
- Field name = **password**

7)

a. Demonstrate how to Send ICMP packets in Scapy and show the packet captured using Wireshark to destination 8.8.8.8.
b. Using NetCat, show communication between browser and the server (for an image).
c. How many items for purchase are in BadStore's database? Use SQL injection on the quick search form field to find out.

(a) Scapy – Send ICMP Packet + Wireshark

1) What is the experiment about

This experiment demonstrates **sending ICMP packets (ping)** using Scapy and capturing them in Wireshark.

From your PDF:

- Scapy can create and send packets
- Packets can be verified in Wireshark

---

2) How to perform (steps)

1. Open terminal
2. Start Wireshark and begin capture
3. Open Python / Scapy
4. Run the ICMP packet code
5. Observe packets in Wireshark

---

3)

i) Program (based on PDF packet creation)

```
fromscapy.allimport*

packet=IP(dst="8.8.8.8")/ICMP()

send(packet)
```

---

ii) Explanation

- `IP(dst="8.8.8.8")` → destination Google DNS
- `ICMP()` → ping packet
- `/` → combine layers
- `send()` → sends packet

✔️ ICMP echo request is generated

---

4) What to show in output

- Wireshark capture showing:
  - ICMP request
  - Source IP
  - Destination: **8.8.8.8**
- Packet details

✅ (b) Netcat – Browser & Server (Image)

1) What is the experiment about

This experiment shows how **Netcat serves an image to browser**.

From your PDF:

- Netcat can send raw data (like image files)

---

2) How to perform (steps)

1. Place image file (example: `image.jpg`)
2. Open terminal
3. Run:

```
nc -l -p 8080 < image.jpg
```

1. Open browser:

```
http://localhost:8080
```

---

3)

i) Command + explanation

- `nc -l -p 8080` → start server
- `< image.jpg` → send image file

- Browser receives file

---

ii) Explanation

- Netcat sends raw image data
- Browser tries to display it
- Needs proper headers for correct rendering

---

4) What to show in output

- Terminal running Netcat
- Browser showing image (or raw data)

✅ (c) BadStore – Number of Items (SQL Injection)

1) What is the experiment about

This experiment uses **SQL Injection** to extract database information.

From your PDF:

- Search field vulnerable to SQL injection
- Can retrieve item count using `COUNT()`

---

2) How to perform (steps)

1. Open BadStore
2. Go to search box
3. Enter payload:

```
x' union select count(itemnum),count(itemnum),count(itemnu
m),count(itemnum) from itemdb --
```

1. Submit query

---

3)

i) Explanation

- `'` → breaks original query
- `union` → combines queries
- `count(itemnum)` → counts items
- `-` → ignores rest

✔️ Extracts total items

---

ii) Final Answer (from PDF)

✅ Number of items = 16

---

4) What to show in output

- Injected query
- Result page showing count
- Value = **16**

8)

a. Demonstrate how to Perform a Port Scan using Scapy tool on target www.example.com.
b. Using Nmap, Scan for vulnerabilities. Also demonstrate how to save results to a text file.
c. What operations are suppliers permitted to do once they have logged into the "suppliers only" area? Use SQL injection to bypass authentication, or find a way to create an account as a supplier.

(a) Scapy – Port Scan (www.example.com)

1) What is the experiment about

This experiment demonstrates **port scanning using Scapy**.

From your PDF:

- Scapy can send packets and analyze responses
- Used for network scanning and analysis

---

2) How to perform (steps)

1. Open terminal
2. Run Python / Scapy
3. Execute port scan script

---

3)

i) Program (based on PDF Scapy usage)

```
fromscapy.allimport*

target="www.example.com"

forportinrange(20,25):
packet=IP(dst=target)/TCP(dport=port,flags="S")
response=sr1(packet,timeout=1,verbose=0)

ifresponse:
print("Port",port,"is open")
```

---

ii) Explanation

- `IP(dst=target)` → destination host
- `TCP(dport=port)` → target port
- `flags="S"` → SYN packet (scan)
- `sr1()` → send & receive one response

✔️ Checks if port is open

---

4) What to show in output

- List of scanned ports
- Open ports displayed

Example:

```
Port 80 is open
```

✅ (b) Nmap – Vulnerability Scan + Save Output

1) What is the experiment about

This experiment uses **Nmap scripts to detect vulnerabilities**.

From your PDF:

- `-script vuln` → vulnerability scanning

---

2) How to perform (steps)

1. Open terminal
2. Run commands

---

3)

i) Commands + explanation

🔹 Vulnerability Scan

```
nmap --script vuln www.example.com
```

✔️ Checks:

- SSL issues
- CVE vulnerabilities
- misconfigurations

🔹 Save Output to File

```
nmap --script vuln www.example.com -oN output.txt
```

✔️ Saves result in text file

---

ii) Explanation

- `-script vuln` → runs vulnerability scripts
- `oN` → normal output file

---

4) What to show in output

- Terminal showing vulnerabilities
- Text file (`output.txt`) with results

✅ (c) BadStore – Supplier Operations

1) What is the experiment about

This experiment focuses on **supplier access in BadStore using SQL injection**.

From your PDF:

- Supplier accounts identified using role = S
- Login using extracted credentials

---

2) How to perform (steps)

1. Perform SQL Injection to get credentials
2. Login as:

   ```
   joe@supplier.com
   ```

3. Go to:

   ```
   supplier portal
   ```

4. Access supplier features

---

3)

i) Answer (from PDF)

👉 Suppliers are allowed to:

✅ Upload Price Lists

---

ii) Explanation

- After login, supplier portal shows options
- Main operation:
  - Upload price lists

---

4) What to show in output

- Supplier login page
- Supplier portal
- Option:

```
Upload Price Lists
```

9)

Using NetCat, show communication between browser and the server (for an image).
b. Using Nmap, Scan a range of IPs. Also scan an entire subnet.
c. Log in as joe@supplier.com — — this is possible in a variety of ways, including SQL injection. Then look at his previous orders and answer the question: What credit card number did he use to make a purchase of $46.95? Multiple answers are possible, but we will accept all of them.

(a) Netcat – Browser & Server Communication (Image)

1) What is the experiment about

This experiment demonstrates how **Netcat can act as a server to send an image to a browser**.

From your PDF:

- Netcat can serve files like images using raw data

---

2) How to perform (steps)

1. Keep an image file (example: `image.jpg`) in your folder
2. Open terminal
3. Run Netcat server:

```
nc-l-p8080 < image.jpg
```

1. Open browser and type:

```
http://localhost:8080
```

---

3)

i) Command + explanation

- `nc -l -p 8080` → listen on port 8080
- `< image.jpg` → sends image file
- Browser connects and receives data

---

ii) Explanation

- Netcat sends raw image data
- Browser tries to display it
- As per PDF:
  - Text works properly
  - Images may need headers

---

4) What to show in output

- Terminal running Netcat
- Browser showing image (or raw output)

✅ (b) Nmap – Scan Range & Subnet

1) What is the experiment about

This experiment uses **Nmap to scan multiple IP addresses**.

From your PDF:

- Supports range scan and subnet scan

---

2) How to perform (steps)

1. Open terminal
2. Run commands

---

3)

i) Commands + explanation

🔹 Scan Range of IPs

```
nmap192.168.1.100-120
```

✔️ Scans IPs from 100 to 120

🔹 Scan Entire Subnet

```
nmap192.168.1.0/24
```

✔️ Scans all 256 IPs

---

ii) Explanation

- Range → specific IPs
- `/24` → full subnet
- Finds:
  - active hosts
  - open ports

---

4) What to show in output

- List of IPs scanned
- Host status
- Open ports

✅ (c) BadStore – Credit Card from Supplier Orders

1) What is the experiment about

This experiment uses **SQL Injection and privilege access** to view sensitive data.

From your PDF:

- Login as supplier using extracted credentials
- View previous orders

---

2) How to perform (steps)

1. Login as admin:

   ```
   admin : secret
   ```

2. Go to:

   ```
   /cgi-bin/badstore.cgi?action=admin
   ```

3. View users → find supplier:

```
joe@supplier.com
```

4. Decode password (MD5 → plain text)
5. Login as supplier
6. Go to **Supplier Portal**
7. Check **Previous Orders**

---

3)

i) Answer (from PDF)

👉 For purchase of **$46.95**:

✅ Credit card number: (Any valid one shown in output)

✔️ PDF clearly states:

- Multiple answers possible
- Any valid number is accepted

---

ii) Explanation

- Orders contain:
  - price
  - credit card details
- You extract the one matching **$46.95**

---

4) What to show in output

- Supplier login
- Orders page
- Order with:

```
$46.95
```

- Credit card number visible

10)

a. Create a simple network monitor that watches for ICMP packets using python function callback in Scapy.
b. Using Nmap, scan the top 1000 ports (default). Also show how to detect service version.
c. Get administrator privileges and then use the admin action to look at the user database. There are two users whose emails have the form XXX@whole.biz; what is the XXX portion of either of the two users? For example, if one of the users is jackie@whole.biz, the right answer is jackie. (The answer is case-sensitive.)

(a) Scapy – Network Monitor (ICMP using callback)

1) What is the experiment about

This experiment creates a **network monitor using Scapy** to detect ICMP packets.

From your PDF:

- Scapy can **sniff packets** and analyze them
- Packet details can be accessed using functions

---

2) How to perform (steps)

1. Open terminal
2. Run Python / Scapy
3. Execute sniffing program

---

3)

i) Program (based on PDF sniff concept)

```
fromscapy.allimport*

defcallback(packet):
ifpacket.haslayer(ICMP):
print("ICMP Packet Detected")
packet.show()

sniff(filter="icmp",prn=callback,count=10)
```

---

ii) Explanation

- `sniff()` → captures packets
- `filter="icmp"` → only ICMP packets
- `prn=callback` → calls function for each packet
- `haslayer(ICMP)` → checks ICMP
- `packet.show()` → displays details

✔️ Works as network monitor

---

4) What to show in output

- "ICMP Packet Detected" message
- Packet details:
  - Source IP
  - Destination IP
  - Type/code

✅ (b) Nmap – Top 1000 Ports & Service Version

1) What is the experiment about

This experiment uses **Nmap scanning features**.

From your PDF:

- Default scan → top 1000 ports
- `sV` → service version detection

---

2) How to perform (steps)

1. Open terminal
2. Run commands

---

3)

i) Commands + explanation

🔹 Default Scan (Top 1000 ports)

```
nmap192.168.1.1
```

✔️ Scans top 1000 ports

🔹 Service Version Detection

```
nmap-sV192.168.1.1
```

✔️ Shows service + version

---

ii) Explanation

- Default → fast scan
- `sV` → detects service versions

---

4) What to show in output

```
PORT    STATE SERVICE VERSION
80      open  http    Apache2.4
```

✅ (c) BadStore – Users with @whole.biz

1) What is the experiment about

This experiment involves:

- **Admin access**
- Viewing **user database**
- Extracting specific user details

From your PDF:

- Admin login gives access to users list
- Emails and passwords visible

---

2) How to perform (steps)

1. Login as admin:

   ```
   admin : secret
   ```

2. Go to:

   ```
   /cgi-bin/badstore.cgi?action=admin
   ```

3. Click **Show Current Users**
4. Look for emails ending with:

   ```
   @whole.biz
   ```

5. Extract username part

---

3)

i) Answer (from PDF data)

👉 The XXX portion can be:

✅ dave *(example from dataset)*

✔️ (Any valid user from @whole.biz is accepted)

---

ii) Explanation

- Email format:

```
XXX@whole.biz
```

- Extract only:

```
XXX
```

---

4) What to show in output

- Admin panel
- User list
- Emails:

```
something@whole.biz
```

- Highlight:

```
XXX
```

11)

a. Identify hosts that are up in a local network using Scapy (hint: ARP)
b. Using NetCat show communication between browser and the server (for an image).
c. BadStore uses cookies to implement a session key, once you've authenticated, and for tracking the contents of the cart, once you've added something to it. You can figure out the cookies in use by BadStore in various ways. What is the key of the session cookie?

(a) Scapy – Identify Hosts (ARP)

1) What is the experiment about

This experiment uses **Scapy with ARP packets** to identify active hosts in a network.

From your PDF:

- ARP is used to discover devices in local network
- Combines Ethernet + ARP for broadcast scanning

---

2) How to perform (steps)

1. Open terminal
2. Run Python script
3. Execute ARP scan

---

3)

i) Program (from PDF)

```
importscapy.allasscapy

request=scapy.ARP()
request.pdst="192.168.1.0/24"

broadcast=scapy.Ether()
broadcast.dst="ff:ff:ff:ff:ff:ff"

request_broadcast=broadcast/request

clients=scapy.srp(request_broadcast,timeout=1)[0]

forelementinclients:
print(element[1].psrc+" - "+element[1].hwsrc)
```

---

ii) Explanation

- `ARP()` → creates ARP request
- `pdst` → target subnet
- `Ether()` → Ethernet frame
- Broadcast → sends to all devices
- `srp()` → send & receive packets

✔️ Returns active hosts (IP + MAC)

---

4) What to show in output

```
192.168.1.1 - xx:xx:xx:xx
192.168.1.2 - xx:xx:xx:xx
```

✔️ List of live hosts

✅ (b) Netcat – Browser & Server (Image)

1) What is the experiment about

This experiment shows **Netcat serving an image to a browser**.

From your PDF:

- Netcat sends raw data (image file)

---

2) How to perform (steps)

1. Keep image file (`image.jpg`)
2. Open terminal
3. Run:

```
nc-l-p8080 < image.jpg
```

1. Open browser:

```
http://localhost:8080
```

---

3)

i) Command + explanation

- `nc -l -p 8080` → listen
- `< image.jpg` → send image
- Browser receives data

---

ii) Explanation

- Netcat acts as server
- Browser acts as client
- Image is transferred

---

4) What to show in output

- Terminal running Netcat
- Browser displaying image

✅ (c) BadStore – Session Cookie Key

1) What is the experiment about

This experiment focuses on **cookies used for authentication and session tracking**.

From your PDF:

- BadStore uses cookies for:
  - session
  - cart tracking

---

2) How to perform (steps)

1. Login into BadStore
2. Open browser → Inspect
3. Go to:
   - Application → Cookies
4. Observe cookie names

---

3)

i) Answer (from experiment)

👉 The **session cookie key** is:

✅ sessionid

---

ii) Explanation

- Session cookie maintains login state
- Used for authentication
- Different from cart cookie

---

4) What to show in output

- Cookie list
- Cookie name:

```
sessionid
```

- Value (encoded)

12)

a. Display the route and measure transit delays of packets across an IP network with Scapy (on www.example.com, www.google.com)
b. Using NetCat, show communication between browser and the server(for an image).

c. BadStore uses cookies to track the contents of the cart, once you've added something to it. What is the key of the cookie used for the cart?

(a) Scapy – Traceroute (Route + Delay)

1) What is the experiment about

This experiment demonstrates **traceroute using Scapy** to:

- Display route (hops)
- Measure delay between nodes

From your PDF:

- Scapy can trace packet path and show responses

---

2) How to perform (steps)

1. Open terminal
2. Start Python / Scapy
3. Run traceroute code

---

3)

i) Program (from PDF)

```
fromscapy.allimport*

result,unanswered=traceroute(["www.example.com","www.googl
e.com"],maxttl=20)

result.show()
```

---

ii) Explanation

- `traceroute()` → finds path to destination
- Targets:

    - www.example.com
    - www.google.com
- `maxttl=20` → max hops
- `result.show()` → displays details

✔️ Shows:

- Each hop IP
- Delay (RTT)
- Route path

---

4) What to show in output

- Hop-by-hop details
- IP addresses
- Delay values

✅ (b) Netcat – Browser & Server (Image)

1) What is the experiment about

This experiment shows **Netcat acting as a server to send an image to browser**.

From your PDF:

- Netcat can serve files like images

---

2) How to perform (steps)

1. Keep image file (`image.jpg`)
2. Open terminal
3. Run:

```
nc-l-p8080 < image.jpg
```

1. Open browser:

```
http://localhost:8080
```

---

3)

i) Command + explanation

- `nc -l -p 8080` → listen
- `< image.jpg` → send image

---

ii) Explanation

- Netcat sends raw image data
- Browser displays it (if supported)

---

4) What to show in output

- Terminal running Netcat
- Browser showing image

✅ (c) BadStore – Cart Cookie Key

1) What is the experiment about

This experiment focuses on **cookies used for cart tracking**.

From your PDF:

- Cart data stored in cookies
- Contains encoded information

---

2) How to perform (steps)

1. Open BadStore
2. Add item to cart
3. Inspect → Application → Cookies

4. Locate cart cookie

---

3)

i) Answer

👉 The key of the cart cookie is:

✅ cart

---

ii) Explanation

- Stores:
  - item
  - quantity
  - price

---

4) What to show in output

- Cookie section
- Cookie name:

  ```
  cart
  ```

- Encoded value